Lab 36: The nightly job hangs because nobody can press Enter

Topic 3: Claude Code Configuration & Workflows · Task 3.6: Integrate Claude Code into CI/CD pipelines
Runs on Simulator 5: Claude Code for Continuous Integration · Run in class
Source: SkillCertPro practice exam Part 1, Q18 (unofficial third-party question)

The question

Your CI job runs a nightly support-triage check and needs Claude Code to process a prompt, write the result to standard output, and exit without waiting for a developer. The current job launches Claude Code and intermittently hangs in the runner. What change best fits this automated workflow?

A. Invoke Claude Code with claude -p (or claude --print) in the job.

B. Launch plain claude and add an instruction telling Claude to exit after answering.

C. Add --output-format json to plain claude and parse the response from stdout.

D. Wrap the interactive claude command in a timeout and retry the job on failure.

Simulate it live

```
# In the browser: open Simulator 5, then Lab 36
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 36

# In a terminal, from simulators/simulator-5-ci-cd/
uv run casekit.py --case UC-18
```

What you should see

One JSON envelope on stdout, and the process exits well inside the job timeout.

claude -p processes the prompt, writes one JSON envelope to stdout and exits well inside the job timeout; no terminal is ever waited on.

Open `simulators/simulator-5-ci-cd/evidence/` and find it yourself.

The answer

A. Run claude -p.

The key distinction is between an interactive session and print mode. claude -p "query" runs the request non-interactively and then exits, which suits scripts and CI/CD. It behaves like a Unix tool, reading stdin and writing stdout. If the job later needs machine-readable output, combine print mode with --output-format json . Structured output does not replace non-interactive mode.

Why the others fail

B. A prompt instruction does not change the CLI's execution mode.

C. Output format does not make an interactive session non-interactive.

D. Timeouts and retries hide the hang and waste runner time. The job still depends on a human-in-the-loop mode.

Check yourself

- the job exited on its own with a JSON envelope (session_id present)
- it returned inside a CI-friendly time budget

- learner can explain why timeouts, retries or 'please exit' instructions do not change the execution mode

Now change one thing

Fix the loop in repo/stock/totals.py and the caller in repo/stock/jobs.py, rerun, and watch the blocking count fall.

Third-party practice material from SkillCertPro, reconstructed from a video transcript in the Tertiary Markdown Vault. Not a real exam item. Wording may differ from the original.